Dockerizing the BookMySeat Website

Implementation and demonstration report
19 October 2026

BookMySeat was containerized and run successfully using Docker Desktop on macOS ARM64. The production Next.js website and a separate PostgreSQL database run under Docker Compose. Both services passed their health checks, and the homepage and public APIs returned HTTP 200.

Project objective

The objective is to demonstrate how an existing full stack website can be packaged into Docker images, connected to a persistent database, started through one Compose project, and verified with observable evidence.

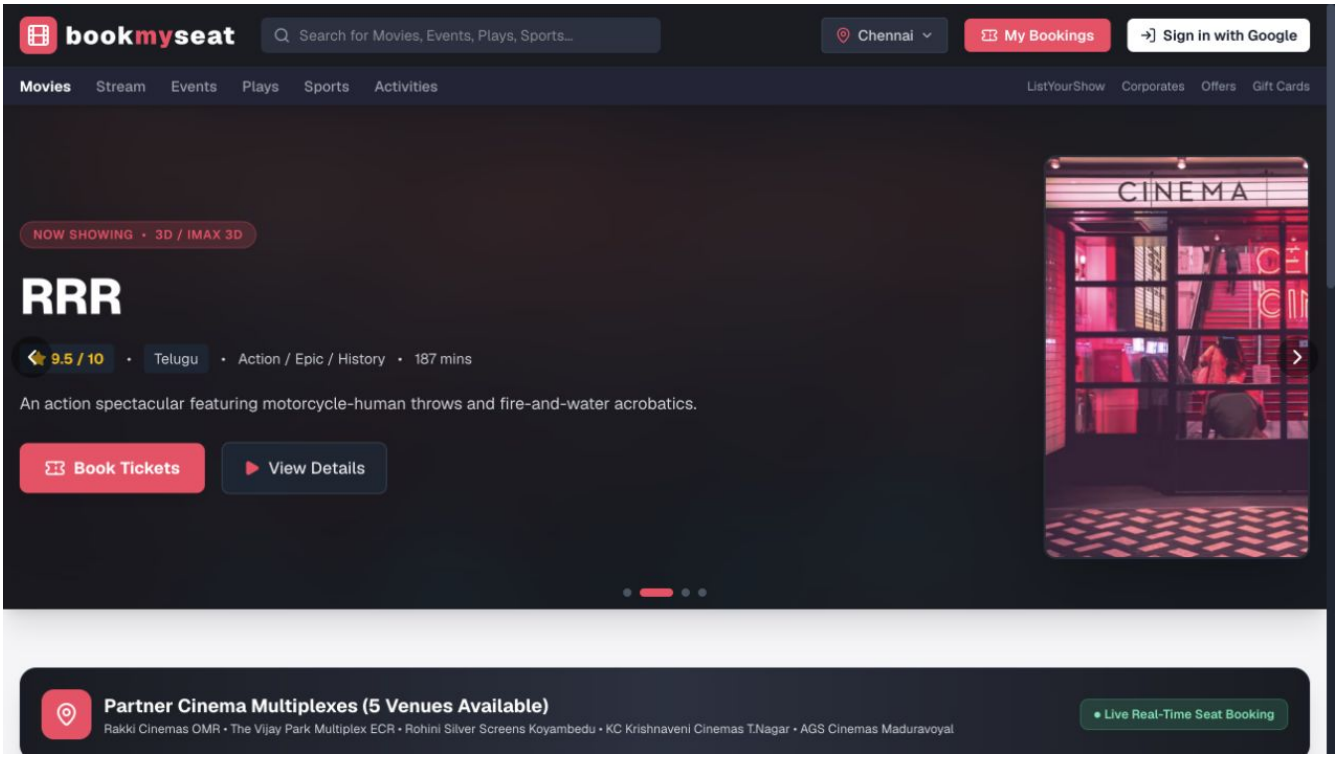


Figure 1. Actual browser capture of BookMySeat served from the Docker web container at http://localhost:3001. The displayed movie data comes from the local Docker database.

Technology: Next.js 16.3.5, React 19, TypeScript, Prisma 6.19.3, PostgreSQL 16, Node.js 22, and Docker Compose. The browser connects to host port 3001, which forwards to port 3000 inside the web container.

Architecture and container evidence

The Compose project is named bookmyseat. The database starts first. The migration service waits for PostgreSQL to become healthy and applies the two committed migrations. The website starts only after migration succeeds. A separate seed command inserts demonstration data.

```
Browser localhost:3001  ->  web container :3000
                                 |
                                 v
                           db container :5432
                                 |
                                 v
                      bookmyseat_postgres_data volume

Startup order: db healthy -> migrate exit 0 -> web
Optional setup: seed -> inserts sample movies and showtimes
```

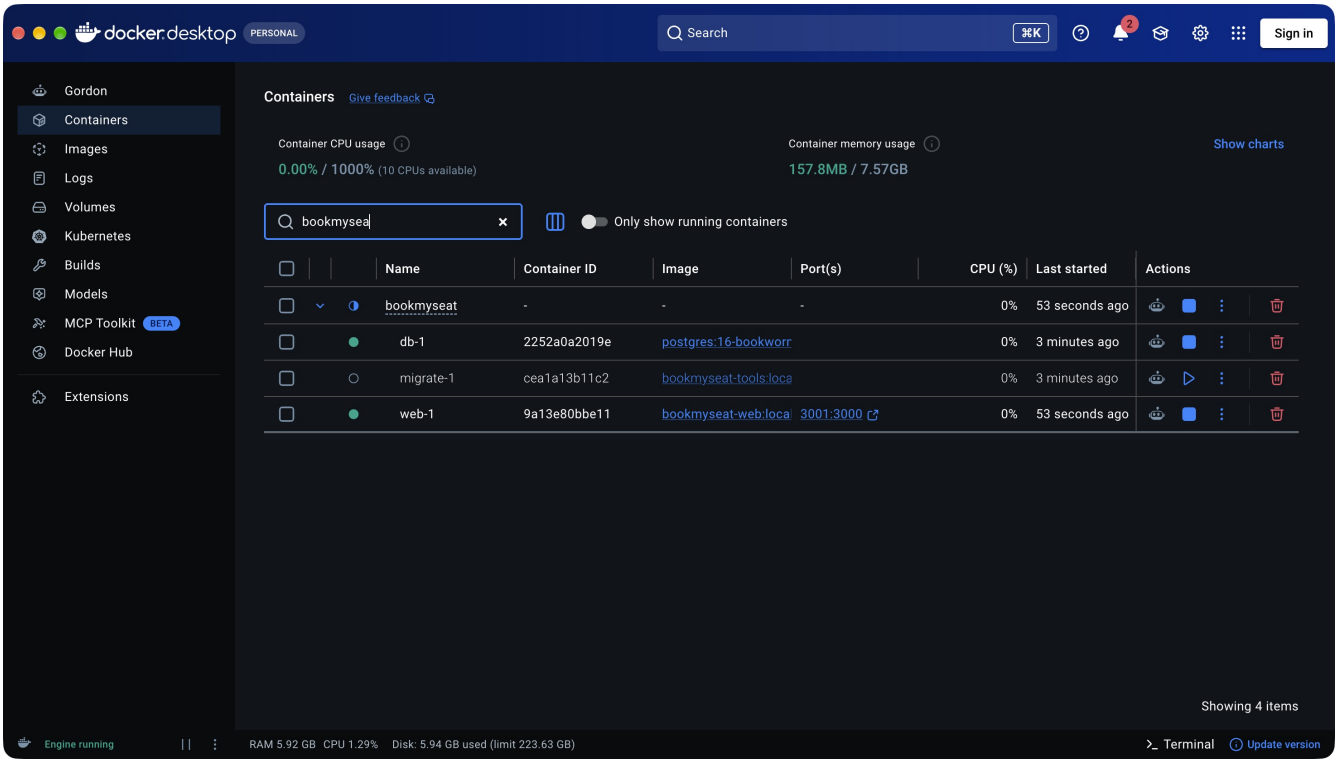


Figure 2. Actual Docker Desktop capture of the expanded bookmyseat Compose project. Green indicators identify running db and web containers. The migration job has completed, and web publishes port 3001 to container port 3000.

Service	Image	Verified state
web	bookmyseat-web:local	Healthy; 127.0.0.1:3001 to 3000
db	postgres:16-bookworm	Healthy; internal port 5432
migrate	bookmyseat-tools:local	Exited with code 0

The migration container is expected to stop after successful completion. It is a one time setup job, while web and db remain running. PostgreSQL is reachable through the Compose network and has no published host port.

Dockerfile implementation

The Dockerfile uses separate dependency, tooling, build, and runtime stages. npm ci installs from the lockfile. Prisma Client is generated in Linux, and Next.js builds standalone output. Only the runtime files and assets are copied into the final web image.

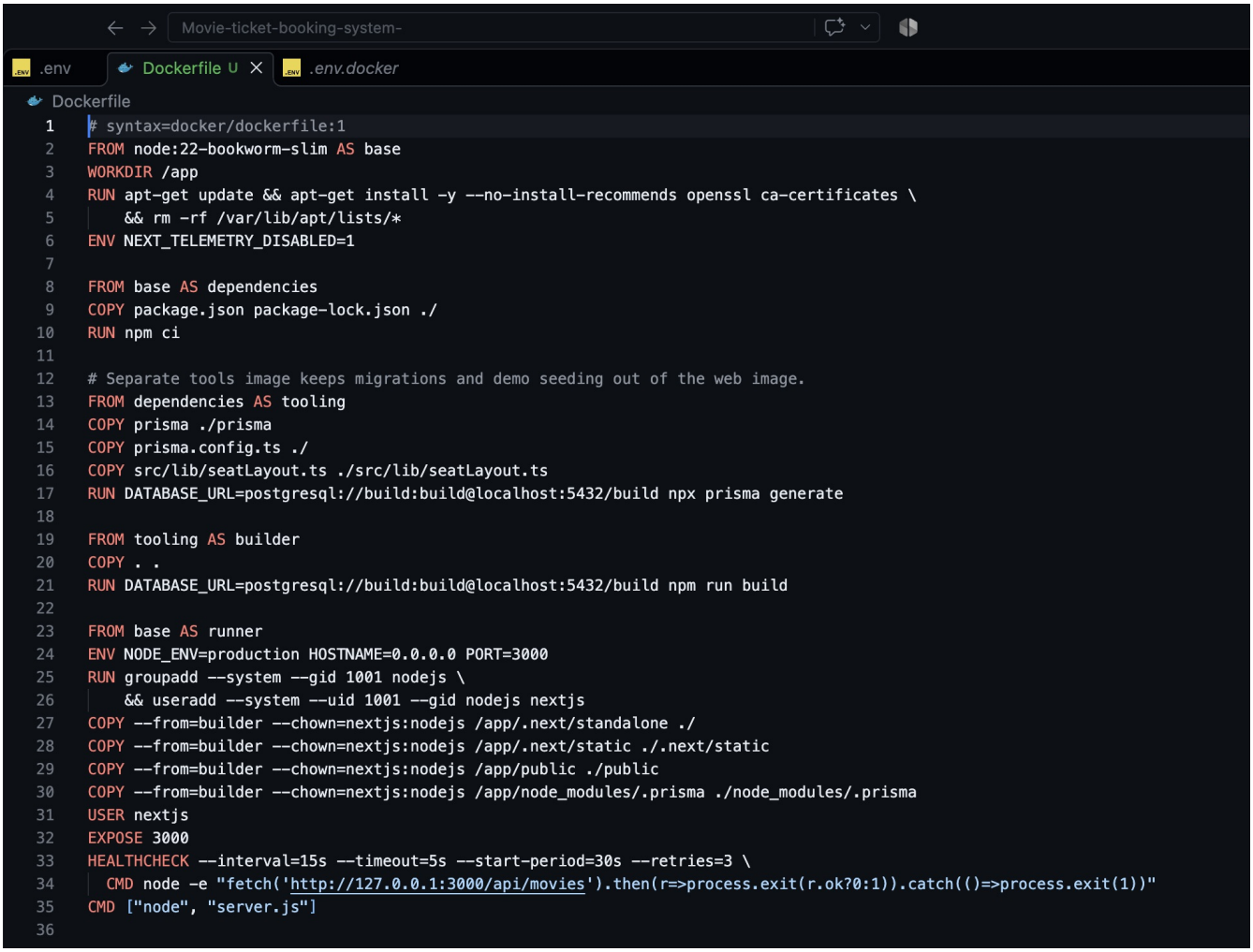


Figure 3. Dockerfile opened in the VS Code editor with syntax highlighting and line numbers. The build stages, non-root user, health check, and server command are visible. Enlarged detail; the full original screenshot is retained in docker-evidence.

Why these choices matter: the runner excludes the full development toolchain, listens on 0.0.0.0 inside the container, and runs as nextjs with UID 1001. OpenSSL supports Prisma on Debian. The health check calls the database backed movies endpoint rather than checking only whether the process exists.

next.config.ts

```
const nextConfig: NextConfig = {
  output: "standalone",
};
```

Compose database and migration configuration

The database uses a named volume so its data survives container replacement. Compose service discovery resolves the hostname db. The connection URL is assembled from the Docker specific password at runtime; credentials are excluded from this report and the build context.

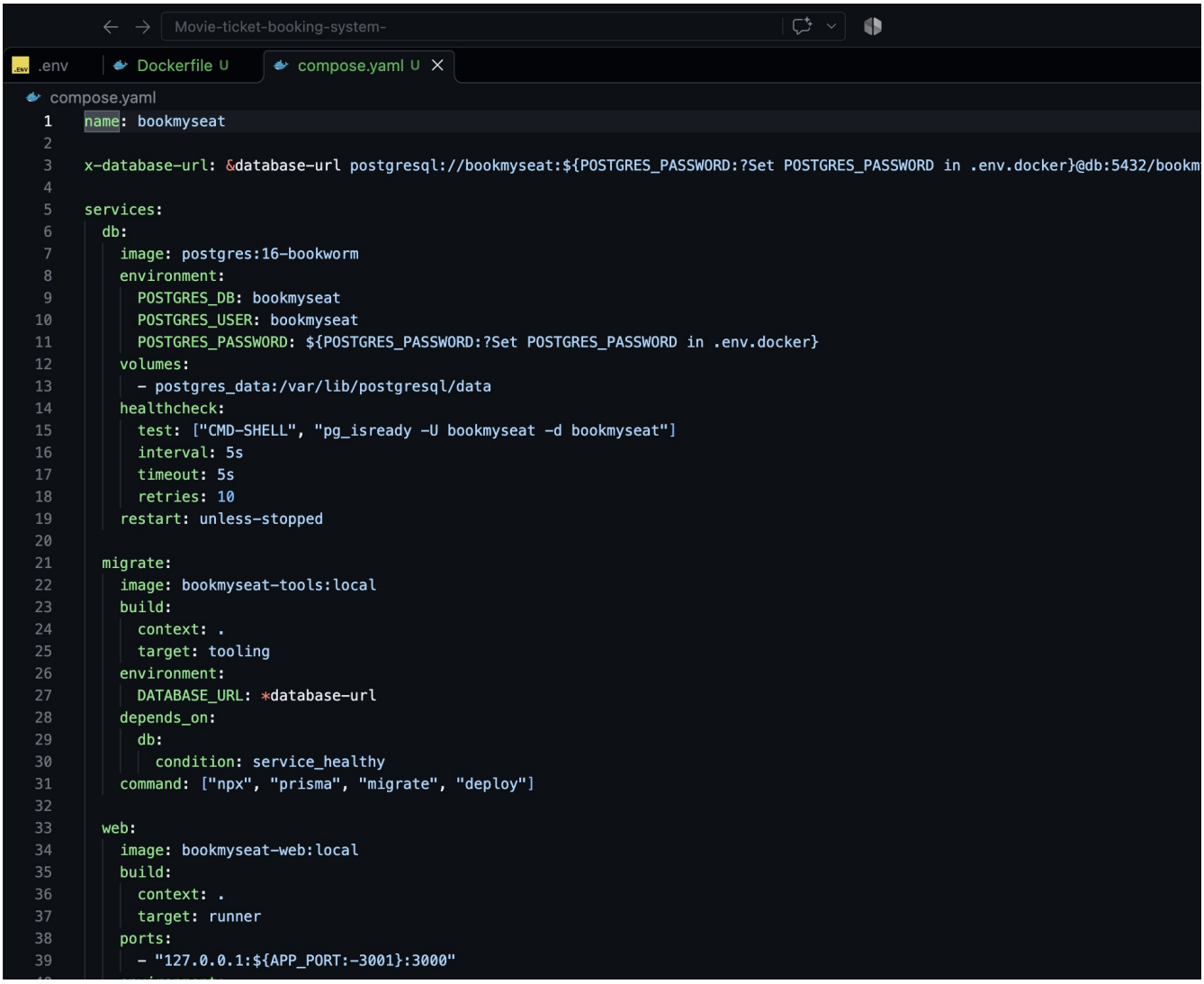


Figure 4. compose.yaml in VS Code showing the database image, persistent volume, readiness check, and migration job. Enlarged detail; the full original screenshot is retained in docker-evidence.

The tooling image includes the Prisma CLI, schema, migrations, and seed script. Separating it from the web image allows database setup without including these tools in the production web runtime.

Build context exclusions

The .dockerignore file excludes node_modules, .next, .git, real .env files, reports, output artifacts, temporary files, screenshots, logs, and local agent folders. Only the safe .env.docker.example template is allowed into the context. This keeps credentials and host build artifacts out of the image.

Compose website and seed configuration

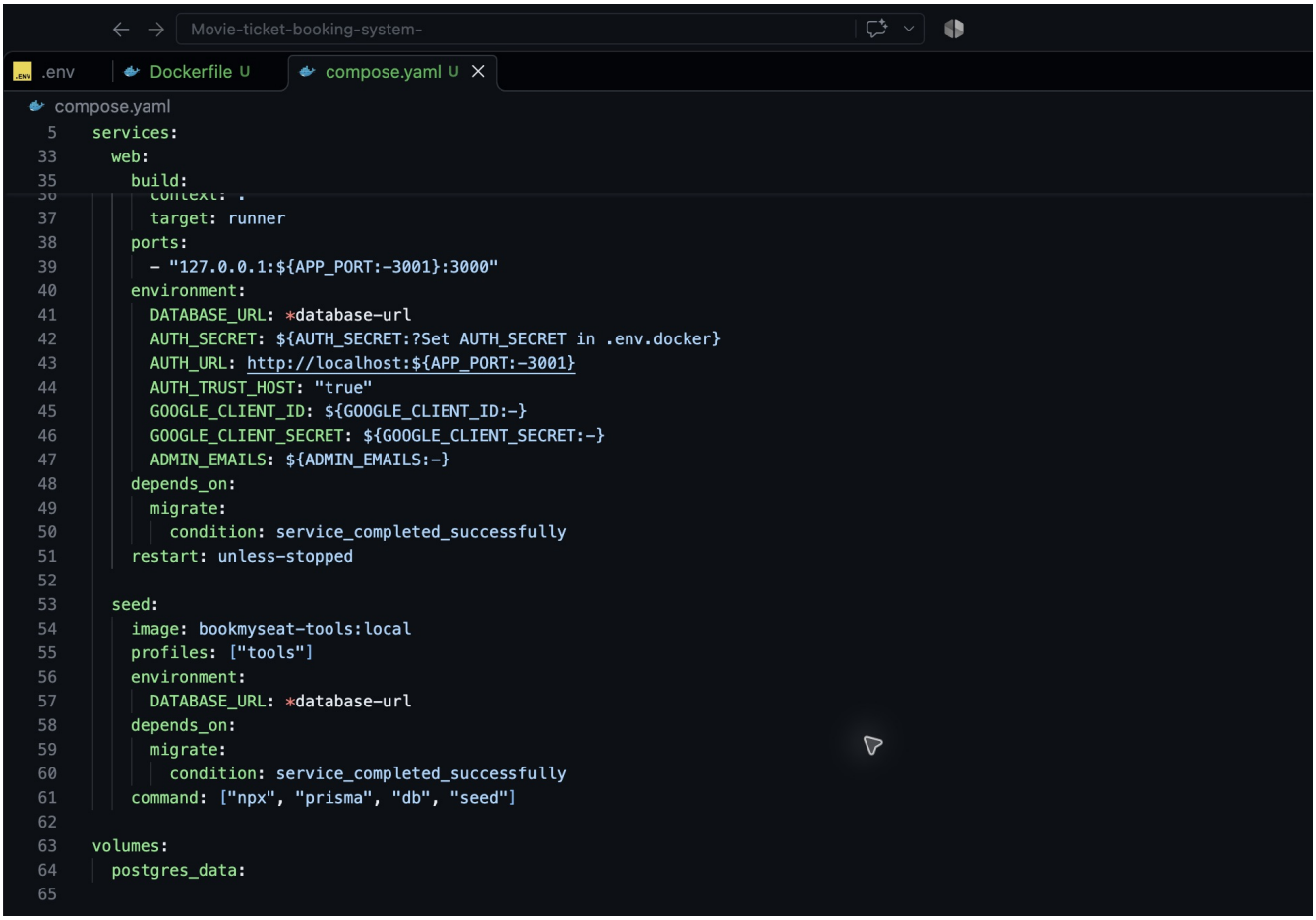


Figure 5. compose.yaml in VS Code showing localhost port publishing, environment variable references, startup dependency, and the optional seed service. Enlarged detail; the full original screenshot is retained in docker-evidence.

APP_PORT defaults to 3001 to avoid colliding with a development server. The host mapping binds to 127.0.0.1 so this demonstration is available locally. AUTH_SECRET and POSTGRES_PASSWORD are required; Google OAuth settings are optional for public browsing.

Environment setup

A separate ignored .env.docker file was created with randomly generated secrets. The committed .env.docker.example documents the required variable names. Keep real credentials out of Git and do not include them in screenshots.

```
POSTGRES_PASSWORD=<random URL safe password>
AUTH_SECRET=<random secret>
APP_PORT=3001
GOOGLE_CLIENT_ID=<optional>
GOOGLE_CLIENT_SECRET=<optional>
ADMIN_EMAILS=<optional>
```

Building and starting from the terminal

The following screenshot was captured directly from the VS Code terminal after running the Docker build and start command. This repeat build reused cached dependency and application layers, exported both images, and confirmed the Compose services were running. The displayed terminal prompt was customized to saidhakshini for these captures.

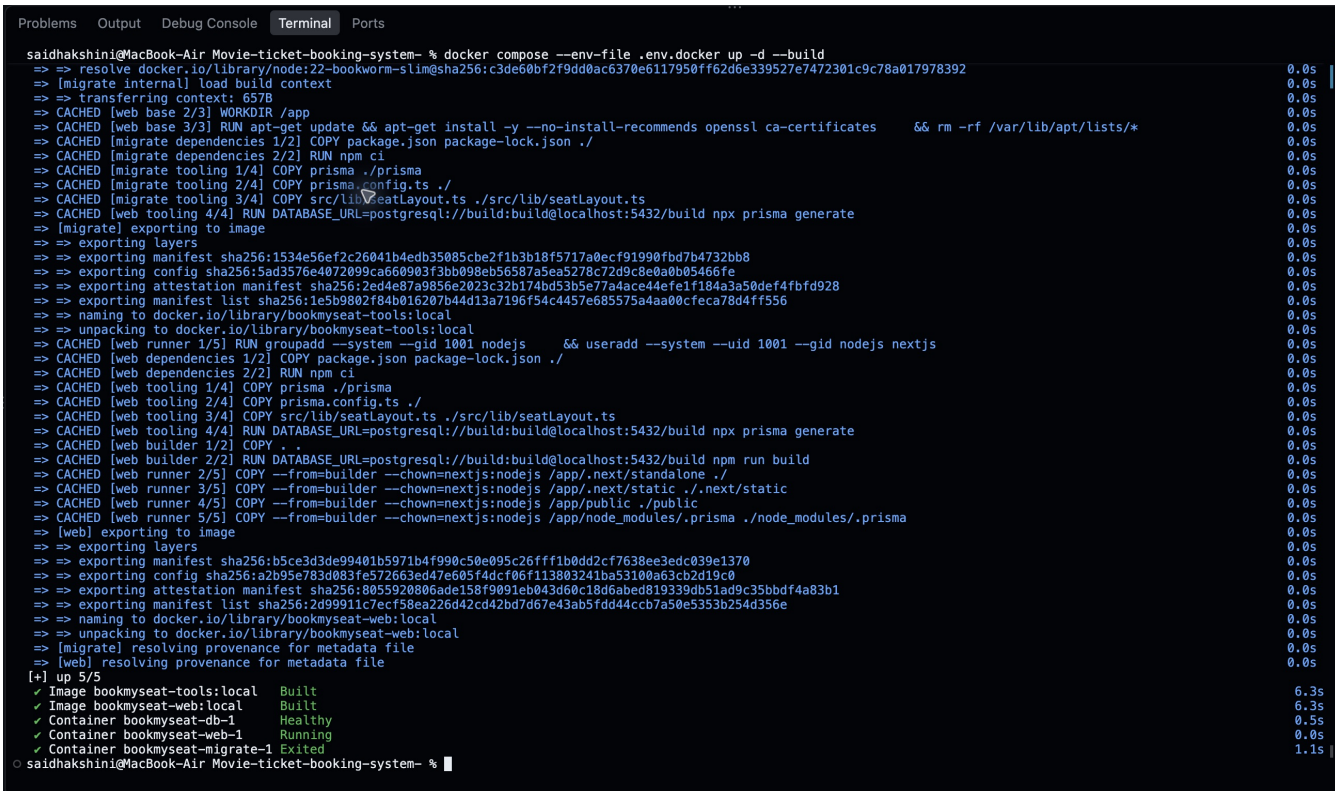


Figure 6. Actual terminal output from docker compose --env-file .env.docker up -d --build. The final status lines show both images built, PostgreSQL healthy, the web container running, and the migration job exited. Enlarged detail; the full original screenshot is retained in docker-evidence.

```
docker compose --env-file .env.docker up -d --build
```

Built confirms that Docker produced the images. Healthy confirms that the database readiness check passed. Started confirms that the web service launched. The migration job is expected to exit after applying migrations. The next screenshot separately verifies the web health check and HTTP response.

Checking the running website from the terminal

These commands inspect the existing containers, request the homepage, read movie records through the public API, and check the operating-system user inside the web container. They were run against the local Docker demonstration.

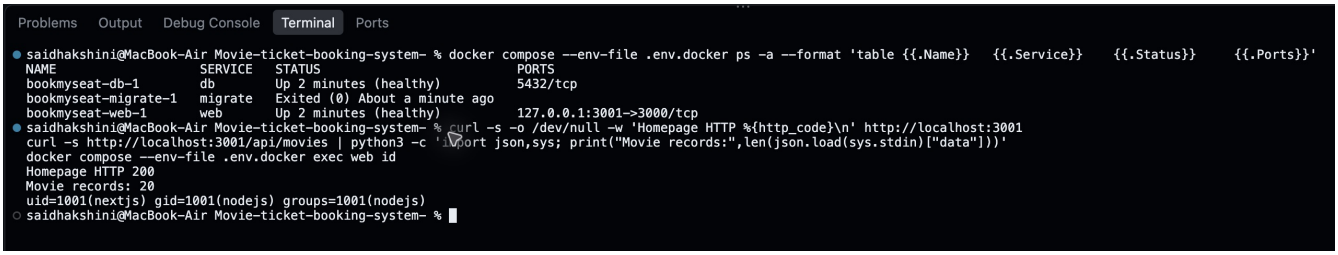


Figure 7. Actual VS Code terminal capture showing 20 movie records, UID 1001 for the runtime user, running healthy web and database services, and Homepage HTTP 200. Enlarged detail; the full original screenshot is retained in docker-evidence.

The number 20 is the length of the movies API data array. The user check returns nextjs with UID 1001, demonstrating that the web process runs without root privileges. The migration job is exited because its setup work is complete.

Inspecting logs and sample data from the terminal

Application startup logs and Prisma migration output can be inspected without entering the containers. The seed service can be invoked as a separate one-off job when sample data is needed.

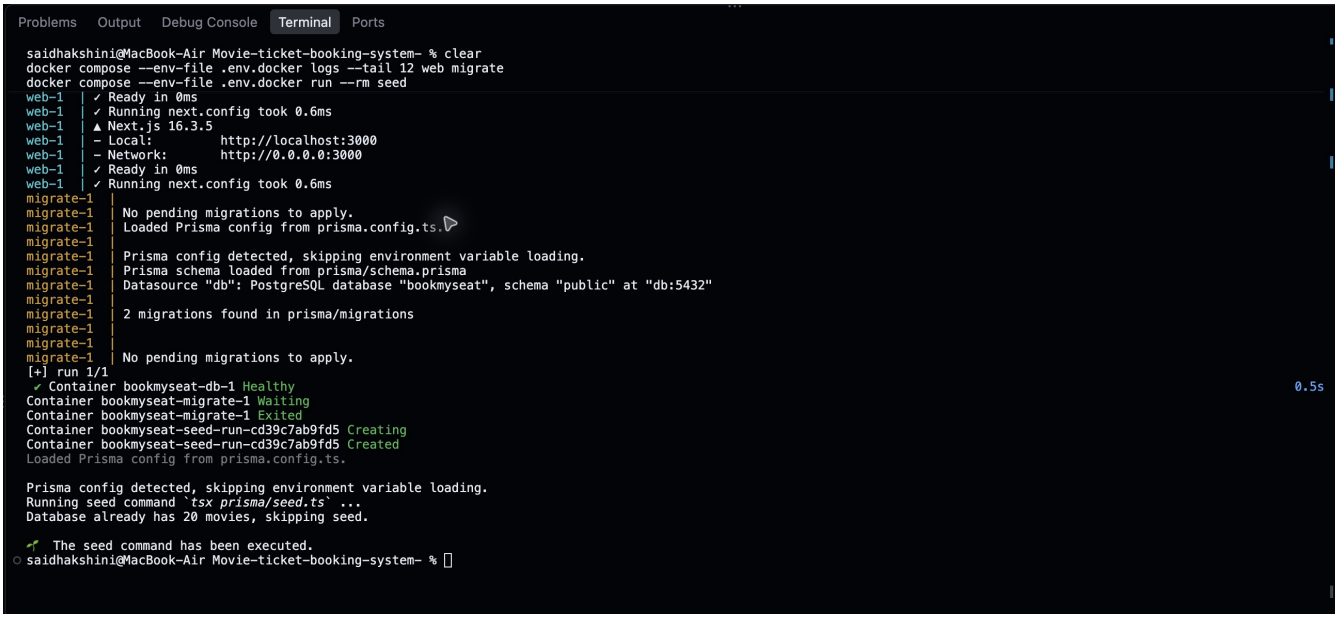


Figure 8. Actual terminal output from the web and migration logs followed by the seed command. Existing sample movies cause the seed script to skip reinsertion. Enlarged detail; the full original screenshot is retained in docker-evidence.

```
docker compose --env-file .env.docker logs --tail 12 web migrate
docker compose --env-file .env.docker run --rm seed
```

No pending migrations to apply confirms that the schema is current. Database already has 20 movies, skipping seed confirms the existing demonstration data is retained and is not inserted again.

Verification results

Check	Observed result
Production image build	Completed; TypeScript and page generation passed
Lint	npm run lint passed
Container health	web healthy and db healthy
Migrations	Two migrations applied; subsequent run found none pending
Homepage	HTTP 200 through localhost:3001
Movies API	HTTP 200; success true; 20 movie records
Theaters API	HTTP 200; 5 theater records
Runtime user	uid=1001(nextjs), gid=1001(nodejs)
Web restart	Homepage and APIs still returned HTTP 200
Database contents	20 movies; 5 theaters; 3500 showtimes; 280000 seats

Recorded startup output

```
web-1 | Next.js 16.3.5
web-1 | Local: http://localhost:3000
web-1 | Network: http://0.0.0.0:3000
web-1 | Ready in 0ms

migrate-1 | 2 migrations found in prisma/migrations
migrate-1 | Applying migration 20260915033625_init
migrate-1 | Applying migration 20260915040016_add_auth
migrate-1 | All migrations have been successfully applied.

After restarting web:
/ HTTP 200 HTML received
/api/movies HTTP 200 records=20
/api/theaters HTTP 200 records=5
```

The web restart confirms the application reconnects to the database and still reads the seeded data. A database container recreation was not part of this check; durability across recreation is provided by the configured named volume.

Evidence files alongside this report retain container status, startup logs, image details, the runtime user check, and the original screenshots. The web image was built for linux/arm64 on this machine. An amd64 deployment requires a build for that architecture.

Reproduce the demonstration

Open Docker Desktop and wait for the engine to report that it is running. From the repository directory, create the Docker environment file, replace the two placeholder secrets, build the images, and load sample data.

```
cp .env.docker.example .env.docker
openssl rand -hex 24  # use for POSTGRES_PASSWORD
openssl rand -hex 32  # use for AUTH_SECRET

docker compose --env-file .env.docker up -d --build
docker compose --env-file .env.docker run --rm seed
docker compose --env-file .env.docker ps -a

# Open http://localhost:3001
curl http://localhost:3001/api/movies
docker compose --env-file .env.docker logs web migrate
docker compose --env-file .env.docker exec web id
```

Present the work

Show Dockerfile and explain the build and runtime stages. Open compose.yaml and explain the startup dependencies, network connection, and persistent volume. In Docker Desktop, expand bookmyseat to show web, db, and the completed migration job. Open localhost:3001, browse the movies, and show the successful API response. Explain why the host and container ports are different.

Stop and resume

```
docker compose --env-file .env.docker stop
docker compose --env-file .env.docker start

# Remove containers while retaining database data
docker compose --env-file .env.docker down
```

Do not add -v to down unless deleting the demonstration database is intended. The sample seed skips insertion when movies already exist.

Scope and limitations

Public browsing and API reads were verified. Google sign-in, authenticated bookings, and admin actions were not tested because the Docker environment does not contain Google OAuth credentials. To enable them, supply the OAuth variables and register http://localhost:3001/api/auth/callback/google. External poster images and build time Google font downloads require internet access.

The dependency installation reported 11 audit findings in the existing dependency tree, including development packages. Dependency remediation and an internet facing deployment were outside this demonstration. HTTPS, backups, production secrets, vulnerability review, and public routing should be addressed before public deployment.

Result: the project now has a working Docker build and Compose setup with an isolated seeded database, demonstrated through Docker Desktop, browser output, health checks, migrations, and HTTP verification.